

FORMATION DÉBUTANT

HTML CSS

CRÉEZ VOS PREMIÈRES
PAGES WEB



LES BASES ESSENTIELLES
POUR COMPRENDRE
ET PROGRESSER



FORMATION INFORMATIQUE

HTML et CSS

Les bases pour construire une page web.
Explique simplement, sans blabla inutile.

DanielCraft - Livre debutant

Sommaire

1. Chapitre 1 - Salut, c'est quoi une page web ?
2. Chapitre 2 - Ton premier fichier HTML
3. Chapitre 3 - Les balises, c'est des etiquettes
4. Chapitre 4 - Titres, textes, paragraphes
5. Chapitre 5 - Liens et images
6. Chapitre 6 - Listes et tableaux
7. Chapitre 7 - Les formulaires
8. Chapitre 8 - CSS : on habille la page
9. Chapitre 9 - Couleurs, polices, tailles
10. Chapitre 10 - Les boites (margin, padding, border)
11. Chapitre 11 - Flexbox : ranger les blocs
12. Chapitre 12 - Un site qui marche sur telephone
13. Chapitre 13 - Mini-projet : ta page perso
14. Chapitre 14 - Ce qu'il faut retenir (+ la suite)
15. Quiz final - Teste-toi !
16. Bravo.

Chapitre 1 - Salut, c'est quoi une page web ?



A gauche la structure. A droite la deco. HTML puis CSS.

Imagine une maison.

Le HTML, c'est les murs, les pieces, les portes. La structure.

Le CSS, c'est la peinture, les rideaux, les couleurs. L'apparence.

Sans HTML, y'a rien a regarder.

Sans CSS, ca marche, mais c'est moche. Genre page blanche avec du texte noir. Pas super sexy.

Le navigateur

Quand tu ouvres Chrome, Firefox, Edge ou Safari, tu lances un navigateur.

C'est lui qui lit ton code et te montre la page.

Tu ecris du texte special (le code).

Le navigateur le transforme en page.

HTML et CSS, deux roles

- HTML dit : "ici un titre", "ici une image", "ici un bouton"
- CSS dit : "ce titre est grand et bleu", "cette image a des bords ronds"

Les deux marchent ensemble. Toujours.

Pourquoi commencer par ca ?

Parce que presque tout le web repose la-dessus.

Les sites, les blogs, les boutiques... Au fond, y'a du HTML et du CSS.

Tu apprendras d'autres trucs plus tard (JavaScript, etc.).

Mais d'abord : les bases. Solides.

Ce que tu vas savoir faire a la fin

- Creer une vraie page web
- Ajouter du texte, des images, des liens

- Changer les couleurs et les polices
- Ranger les blocs proprement
- Faire une petite page perso

Pas besoin d'être un génie.

Juste de la curiosité et un peu de patience.

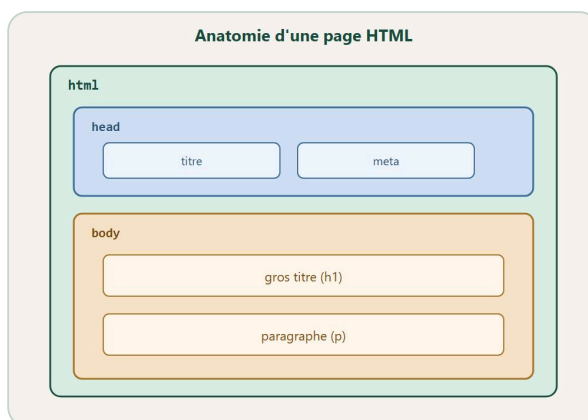
Allez, on attaque.

CHAPITRE 2

Chapitre 2 - Ton premier fichier HTML



Ton fichier s'ouvre dans le navigateur. Magie.



La page, c'est des boîtes rangées les unes dans les autres.

On va créer un fichier. Rien de compliqué.

De quoi tu as besoin

1. Un éditeur de texte. Bloc-notes marche. VS Code c'est mieux (gratuit).
2. Un navigateur (celui que tu utilises déjà).

C'est tout. Pas de compte. Pas de carte bleue.

Crée le fichier

1. Ouvre ton éditeur.
2. Copie le code ci-dessous.
3. Enregistre le fichier en `index.html` (attention à l'extension `.html`).

4. Double-clique dessus. Ca s'ouvre dans le navigateur.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Ma premiere page</title>
</head>
<body>
  <h1>Salut !</h1>
  <p>Ca y est. J'ai fait une page web.</p>
</body>
</html>
```

Si tu vois "Salut !" en gros, bravo. C'est bon.

On explique ligne par ligne (sans stress)

```
<!DOCTYPE html>
```

Ca dit au navigateur : "hey, c'est du HTML moderne". Toujours en haut.

```
<html lang="fr">
```

La page commence. `lang="fr"` = contenu en francais.

```
<head> ... </head>
```

La tete de la page. Infos pour le navigateur, pas vraiment ce que tu vois au milieu de l'ecran.

Exemple : le titre de l'onglet (`<title>`).

```
<meta charset="UTF-8">
```

Pour que les accents marchent. Crucial en francais. Sans ca, tu peux avoir des caracteres bizarres.

```
<body> ... </body>
```

Le corps. Tout ce que tu vois sur la page va ici.

```
<h1>
```

 = un gros titre.

```
<p>
```

 = un paragraphe.

Les balises ouvrantes et fermantes

Regarde : `<p>` ouvre. `</p>` ferme.

Le slash `/` veut dire "c'est fini pour ce truc".

Comme des parentheses. Tu ouvres, tu fermes.

A toi

Change le texte. Mets ton prenom dans le titre.

Rafraichis la page (F5).

Tu viens de modifier un site. Serieux.

Chapitre 3 - Les balises, c'est des etiquettes



Une balise, c'est une etiquette posee sur un contenu.

Une balise, c'est une etiquette.

Elle dit au navigateur : "ce bout de texte, c'est un titre" ou "c'est un lien".

Forme generale

```
<nom>contenu</nom>
```

Exemple :

```
<strong>Important</strong>
```

Ca met le mot en gras.

Quelques balises utiles tout de suite

Balise	Role simple
<code>h1</code> a <code>h6</code>	Titres (h1 le plus grand)
<code>p</code>	Paragraphe
<code>a</code>	Lien
<code>img</code>	Image
<code>ul</code> / <code>ol</code> / <code>li</code>	Listes
<code>div</code>	Boite generique (on range des trucs dedans)
<code>span</code>	Petit bout de texte dans une phrase
<code>br</code>	Retour a la ligne (balise seule)

Balises seules (pas de fermeture)

Certaines n'ont pas de contenu à enfermer.

```
<br>

```

`br` = aller à la ligne.

`img` = afficher une image.

Attributs : des infos en plus

Dans une balise, tu peux ajouter des détails.

```
<a href="https://exemple.com">Clique ici</a>
```

`href` = l'adresse du lien.

`src` = le chemin de l'image.

`alt` = texte si l'image ne charge pas (utile aussi pour l'accessibilité).

HTML, c'est de l'ordre

Le navigateur lit de haut en bas.

Ce que tu mets en premier apparaît en premier (sauf si le CSS change l'ordre plus tard).

Erreur classique

Oublier de fermer une balise.

Ou écrire `</p>` sans avoir ouvert `<p>`.

Si la page a l'air cassée, regarde d'abord ça.

Respire. Corrige. Rafraîchis.

Mini exo

Écris une page avec :

- un titre `h1`
- deux paragraphes
- un mot en gras avec `<strong>`

Si ça s'affiche, tu as compris l'idée des balises.

CHAPITRE 4

Chapitre 4 - Titres, textes, paragraphes

Le texte, c'est le cœur de presque toutes les pages.

Les titres : h1 à h6

```
<h1>Le plus grand</h1>
<h2>Un cran en dessous</h2>
```

```
<h3>Encore un peu plus petit</h3>
```

Règle simple :

- Un seul `h1` par page (le sujet principal)
- Ensuite `h2` pour les parties
- `h3` pour les sous-parties

C'est comme un plan de rédaction.

Ca aide aussi Google à comprendre ta page. Mais surtout : ça aide le lecteur.

Les paragraphes

```
<p>Une idée par paragraphe, c'est souvent plus clair.</p>
<p>Tu peux en mettre plusieurs à la suite.</p>
```

Ne mets pas tout dans un seul `<p>` géant.

Coupe. Respire. Lis à voix haute si besoin.

Gras, italique, et cie

```
<p>Voici un mot <strong>important</strong>.</p>
<p>Voici un mot <em>en emphase</em> (souvent en italique).</p>
<p>Un <mark>surlignage</mark> pour attirer l'œil.</p>
```

`strong` = vraiment important.

`em` = on insiste un peu.

Évite de tout mettre en gras. Sinon plus rien ne ressort.

Citations et code

```
<blockquote>
  Une citation un peu longue.
</blockquote>

<p>En ligne, on peut citer <code>du code</code>.</p>
```

`code` est pratique quand tu expliques une balise dans une phrase.

Commentaires (pour toi, pas pour le visiteur)

```
<!-- Ceci ne s'affiche pas sur la page -->
```

Utile pour te laisser des notes. Ou désactiver un bout de code sans l'effacer.

À éviter

- Utiliser `h1` juste parce que "c'est plus gros" (utilise le CSS pour la taille)
- Sauter de `h1` à `h4` sans raison
- Des murs de texte sans retour

A toi

Ecris une mini page "Ma journée" :

- h1 avec le titre
- 3 h2 (matin, après-midi, soir)
- un petit paragraphe sous chaque h2

CHAPITRE 5

Chapitre 5 - Liens et images

Sans liens, le web serait juste des pages isolées.

Sans images, ce serait un peu triste.

Les liens

```
<a href="https://danielcraft.fr">Aller sur DanielCraft</a>
```

`a` comme "ancree" (anchor en anglais).

`href` = ou ça mène.

Lien vers une autre page de ton site

```
<a href="contact.html">Contact</a>
```

Lien qui ouvre un nouvel onglet

```
<a href="https://exemple.com" target="_blank" rel="noopener">Site externe</a>
```

`target="_blank"` = nouvel onglet.

`rel="noopener"` = petite sécurité. Prends le réflexe.

Lien mailto

```
<a href="mailto:salut@exemple.com">M'écire</a>
```

Ça ouvre le mail (si le visiteur a un logiciel mail configuré).

Les images

```

```

`src` = où est le fichier image.

`alt` = description. Toujours. Pour ceux qui ne voient pas l'image, et pour le référencement.

Où mettre tes images ?

Le plus simple au début :

```
mon-site/  
  index.html  
  images/  
    chat.jpg
```

Alors :

```

```

Largeur (debutant)

```

```

Plus tard, tu gereras ca mieux avec le CSS. Pour l'instant, c'est ok.

Figure + legende (plus propre)

```
<figure>  
    
  <figcaption>Mon chat, Roi du canape.</figcaption>  
</figure>
```

A toi

1. Ajoute un lien vers une page que tu aimes.
2. Ajoute une image (une photo de chez toi, un dessin, peu importe).
3. Ecris un vrai `alt` .

Si le lien marche et l'image s'affiche, nickel.

CHAPITRE 6

Chapitre 6 - Listes et tableaux

Des listes partout. Courses, etapes, menus...

Des tableaux quand tu compares des infos.

Liste a puces

```
<ul>  
  <li>Pain</li>  
  <li>Lait</li>  
  <li>Beurre</li>  
</ul>
```

`ul` = unordered list (liste sans ordre strict).

`li` = list item (un element).

Liste numerotee

```
<ol>  
  <li>Ouvrir l'editeur</li>
```

```
<li>Ecrire le HTML</li>
<li>Sauvegarder</li>
<li>Ouvrir dans le navigateur</li>
</ol>
```

`ol` = ordered list. Utile pour des etapes.

Liste dans une liste

```
<ul>
  <li>Fruits
    <ul>
      <li>Pommes</li>
      <li>Bananes</li>
    </ul>
  </li>
  <li>Legumes</li>
</ul>
```

Ca marche. Mais n'en abuse pas, sinon ca devient illisible.

Les tableaux

```
<table>
  <thead>
    <tr>
      <th>Langage</th>
      <th>Role</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>HTML</td>
      <td>Structure</td>
    </tr>
    <tr>
      <td>CSS</td>
      <td>Style</td>
    </tr>
  </tbody>
</table>
```

- `table` = le tableau
- `tr` = une ligne
- `th` = cellule d'en-tete
- `td` = cellule normale

Quand utiliser un tableau ?

Quand les donnees sont vraiment en lignes et colonnes.

Pas pour "faire un layout" de page. Ca, c'est le job du CSS.

A toi

Fais une liste de 5 choses que tu veux apprendre.

Puis un petit tableau : jour / activite (ex. Lundi / HTML).

CHAPITRE 7

Chapitre 7 - Les formulaires

Un formulaire, c'est quand tu demandes des infos a quelqu'un.

Nom, email, message... Tu vois le genre.

La base

```
<form action="#" method="post">
  <label for="prenom">Prenom</label>
  <input id="prenom" name="prenom" type="text">

  <button type="submit">Envoyer</button>
</form>
```

`form` enveloppe tout.

`label` = le texte a cote du champ (important pour la clarte).

`input` = le champ.

`button` = le bouton.

`for` du label doit matcher `id` de l'input. Comme ca, cliquer sur le texte focus le champ. Pratique.

Types d'input utiles

```
<input type="text" placeholder="Ton prenom">
<input type="email" placeholder="toi@exemple.com">
<input type="password">
<input type="number" min="1" max="120">
<input type="checkbox"> J'accepte
<input type="radio" name="choix" value="a"> Option A
<input type="radio" name="choix" value="b"> Option B
```

`placeholder` = texte fantome dans le champ. Disparait quand tu ecris.

Zone de texte longue

```
<label for="msg">Message</label>
<textarea id="msg" name="msg" rows="5"></textarea>
```

Liste deroulante

```
<label for="ville">Ville</label>
<select id="ville" name="ville">
  <option value="paris">Paris</option>
  <option value="lyon">Lyon</option>
  <option value="marseille">Marseille</option>
</select>
```

Requis

```
<input type="email" required>
```

Le navigateur refuse d'envoyer si c'est vide. Utile, mais ce n'est pas une vraie securite cote serveur. Pour plus tard.

Petit rappel

Pour l'instant, `action="#"` veut dire "on n'envoie nulle part de special".

Plus tard tu brancheras ca a un vrai traitement (PHP, API...).

La on apprend juste a construire le formulaire.

A toi

Fais un formulaire "Contact" avec :

- nom
- email
- message
- bouton Envoyer

Chapitre 8 - CSS : on habille la page

La le HTML est nu. On va lui mettre des habits.

Trois facons d'ajouter du CSS

1. Dans le fichier HTML (balise style)

```
<head>
  <style>
    p {
      color: blue;
    }
  </style>
</head>
```

Pratique pour tester. Moins propre sur un gros site.

2. Fichier CSS separe (le mieux)

style.css :

```
p {
  color: blue;
}
```

Dans le HTML :

```
<link rel="stylesheet" href="style.css">
```

Mets le `<link>` dans le `<head>`.

3. En ligne (a eviter souvent)

```
<p style="color: blue;">Coucou</p>
```

Ca marche. Mais ca devient vite le bordel. Garde ca pour des cas rares.

Anatomie d'une regle CSS

```
selecteur {
  propriete: valeur;
}
```

Exemple :

```
h1 {
  color: tomato;
  font-size: 40px;
}
```

Le selecteur dit "qui".

Les proprietes disent "quoi changer".

Commentaire CSS

```
/* Ceci est une note pour toi */
```

Premier test

1. Cree `style.css` a cote de `index.html`
2. Relie-les avec `<link>`
3. Mets tous les `p` en bleu
4. Rafraichis

Si c'est bleu, CSS est branche. Nice.

CHAPITRE 9

Chapitre 9 - Couleurs, polices, tailles

Maintenant on rend ca vivant.

Couleurs

```
body {  
  color: #222222;      /* texte */  
  background: #f7f3ee; /* fond */  
}  
  
h1 {  
  color: teal;  
}
```

Tu peux ecrire :

- un nom (`red` , `teal` , `navy`)
- un code hex (`#ff6600`)
- du rgb (`rgb(255, 100, 50)`)

Hex, c'est le plus courant sur le web.

`#000000` = noir. `#ffffff` = blanc.

Astuce : des outils comme un color picker t'aident a choisir. Pas besoin de retenir les codes par coeur.

Polices

```
body {  
  font-family: Georgia, "Times New Roman", serif;  
  font-size: 18px;  
  line-height: 1.6;  
}
```

`font-family` = la police.

Si la premiere n'existe pas sur l'ordi, il prend la suivante.

`line-height: 1.6` = respirer entre les lignes. Plus agreable a lire.

Polices "safe" pour commencer

- `Arial, sans-serif` (moderne, sans empattement)
- `Georgia, serif` (un peu plus litteraire)

Plus tard tu pourras charger des polices web (Google Fonts, etc.). Pas obligatoire au debut.

Graisse et style

```
h1 {  
  font-weight: 700; /* gras */  
}  
  
em {  
  font-style: italic;  
}
```

Taille du texte

```
h1 { font-size: 2.5rem; }  
p { font-size: 1rem; }
```

`px` = pixels. Simple.

`rem` = relatif a la taille de base. Plus souple pour l'accessibilite.

Au debut, `px` c'est ok. Tu passeras a `rem` quand tu seras a l'aise.

Centrer un texte

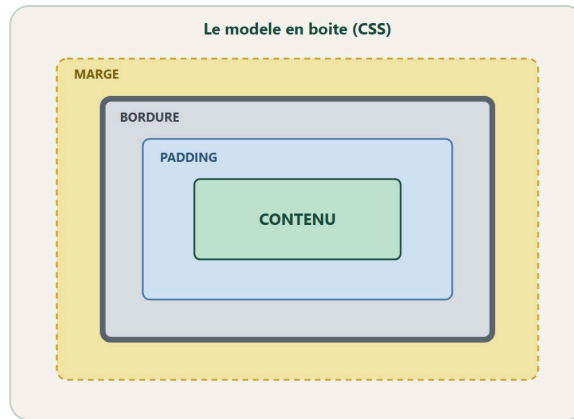
```
h1 {  
  text-align: center;  
}
```

`left`, `right`, `center`, `justify`.

A toi

- Fond doux (pas blanc crame si tu veux etre cosy)
- Texte sombre pour bien lire
- Titre colore
- Paragraphes un peu plus grands (16-18px minimum)

Chapitre 10 - Les boîtes (margin, padding, border)



Margin dehors, padding dedans, border entre les deux.

En CSS, presque tout est une boîte.

Un paragraphe = une boîte. Une image = une boîte. Un bouton = une boîte.

Le modèle en boîte

De l'extérieur vers l'intérieur :

1. margin (marge extérieure - l'espace autour)
2. border (la bordure)
3. padding (marge intérieure - l'air dans la boîte)
4. le contenu

Exemple concret

```
.carte {
  background: white;
  border: 2px solid #333;
  padding: 20px;
  margin: 16px;
  border-radius: 12px;
}
```

`border-radius` arrondit les coins. Sympa.

Margin vs padding (le piège classique)

- padding = espace *dans* la boîte (entre le bord et le texte)
- margin = espace *hors* de la boîte (entre cette boîte et les autres)

Tu te tromperas. Tout le monde se trompe. Tu corrigeras.

Largeur et hauteur

```
.boite {
  width: 300px;
  max-width: 100%;
}
```

```
height: auto;
}
```

`max-width: 100%` évite que ça déborde sur téléphone. Bon réflexe.

box-sizing (important)

```
* {
  box-sizing: border-box;
}
```

Sans ça, padding + border peuvent agrandir la boîte de façon bizarre.

Avec `border-box`, width inclut padding et border. Plus intuitif.

Mets ça presque toujours en haut de ton CSS.

Display : block ou inline

- `block` : prend toute la largeur (comme `p`, `div`, `h1`)
- `inline` : reste dans le flux du texte (comme `span`, `a` sans style)
- `inline-block` : un mix utile parfois

```
span.badge {
  display: inline-block;
  padding: 4px 8px;
  background: #eee;
}
```

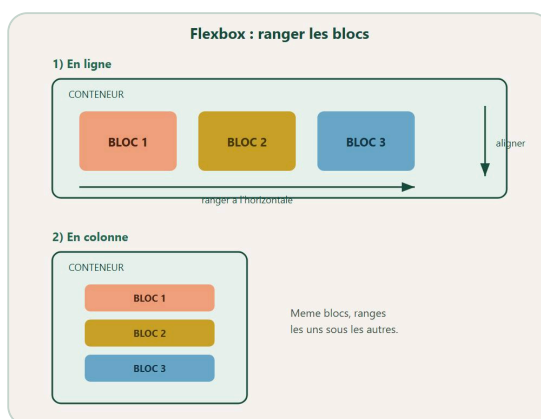
A toi

Crée une classe `.carte` avec fond, padding, bordure, coins ronds.

Applique-la à une `div` qui contient un titre et un paragraphe.

CHAPITRE 11

Chapitre 11 - Flexbox : ranger les blocs



Flexbox range les blocs en ligne ou en colonne.

Flexbox, c'est l'outil magique pour aligner des trucs.

Menu en ligne, colonnes, centrage... Ça sauve des vies.

Idee simple

Tu as un parent (le conteneur).

Dedans, des enfants (les items).

Tu dis au parent : "range tes enfants en flex".

```
<div class="rangée">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

Attends - apostrophe. Mieux :

```
<div class="rangée">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

```
.rangée {
  display: flex;
  gap: 16px;
}
```

`gap` = l'espace entre les enfants. Clean.

Direction

```
.rangée {
  display: flex;
  flex-direction: row; /* cote a cote (default) */
}

.colonne {
  display: flex;
  flex-direction: column; /* l'un sous l'autre */
}
```

Aligner

```
.rangée {
  display: flex;
  justify-content: space-between; /* horizontal si row */
  align-items: center; /* vertical si row */
}
```

`justify-content` = sur l'axe principal.

`align-items` = sur l'axe croise.

Tu vas experimenter. C'est normal. Change une valeur, regarde, recommence.

Centrer un truc au milieu (classique)

```
.centre {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 200px;  
}
```

Les enfants peuvent grandir

```
.rangee > div {  
  flex: 1; /* partage l'espace equitablement */  
}
```

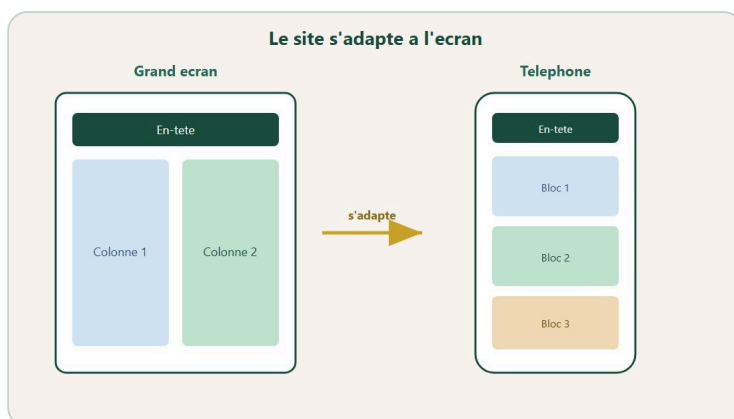
A toi

Fais une barre avec un logo a gauche et deux liens a droite.

Indice : `display: flex` + `justify-content: space-between` + `align-items: center`.

CHAPITRE 12

Chapitre 12 - Un site qui marche sur telephone



Meme site, grand ecran ou telephone : ca s'adapte.

La plupart des gens regardent le web sur mobile.

Si ta page est illisible sur telephone, c'est rate.

La balise magique (viewport)

Dans le `<head>` :

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Sans ca, le telephone croit que ta page fait la largeur d'un grand ecran et zoom de facon horrible.

Mets-la. Toujours.

Largeurs souples

```
img {
  max-width: 100%;
  height: auto;
}

.conteneur {
  width: min(100% - 2rem, 700px);
  margin-inline: auto;
}
```

Traduction : la boîte ne dépasse pas 700px, et sur petit écran elle prend la place dispo avec un peu d'air.

Media queries

```
.menu {
  display: flex;
  gap: 1rem;
}

@media (max-width: 600px) {
  .menu {
    flex-direction: column;
  }
}
```

En dessous de 600px de large, le menu passe en colonne.

Tu adaptes. Tu ne fais pas "un autre site". Tu ajustes.

Tailles de doigt

Sur mobile, les boutons trop petits c'est l'enfer.

Vise une zone cliquable confortable (genre 40px+ de haut).

Teste vraiment

- Reduis la fenêtre de ton navigateur
- Ou ouvre les outils développeur (F12) et le mode téléphone
- Ou regarde sur ton vrai téléphone (en local c'est plus technique, on verra plus tard)

Checklist mobile

- meta viewport présente
- images qui ne débordent pas
- texte lisible sans zoomer
- boutons assez grands
- pas de scroll horizontal bizarre

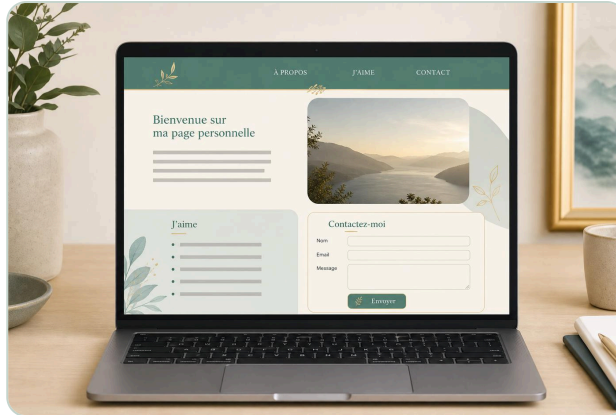
A toi

Prends ta page et ajoute le viewport.

Passe le menu en colonne sous 600px.

Verifie en redimensionnant la fenetre.

Chapitre 13 - Mini-projet : ta page perso



Le but : une petite page a toi, claire et perso.

On assemble tout. Une page sur toi (ou un perso invente, libre a toi).

Objectif

Une page unique `index.html` + `style.css` avec :

- un en-tete (nom + petit menu)
- une section "A propos"
- une section "Ce que j'aime" (liste)
- une image
- un petit formulaire de contact
- un pied de page

Structure HTML proposee

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Ma page perso</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="entete">
    <strong>Ton Prenom</strong>
    <nav class="menu">
      <a href="#apropos">A propos</a>
      <a href="#likes">J'aime</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <main class="conteneur">
    <section id="apropos">
      <h1>Salut, moi c'est ...</h1>
      <p>Deux ou trois phrases sur toi.</p>
      
    </section>
```

```

<section id="likes">
  <h2>Ce que j'aime</h2>
  <ul>
    <li>...</li>
    <li>...</li>
    <li>...</li>
  </ul>
</section>

<section id="contact">
  <h2>Me contacter</h2>
  <form action="#" method="post">
    <label for="nom">Nom</label>
    <input id="nom" name="nom" type="text" required>

    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>

    <label for="msg">Message</label>
    <textarea id="msg" name="msg" rows="4"></textarea>

    <button type="submit">Envoyer</button>
  </form>
</section>
</main>

<footer class="pied">
  <p>Fait avec HTML et CSS. Et un peu de cafe.</p>
</footer>
</body>
</html>

```

CSS de démarrage (a personnaliser)

```

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Georgia, serif;
  font-size: 18px;
  line-height: 1.6;
  color: #222;
  background: #f4f1ec;
}

.entete {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 1.5rem;
  background: #1f6f5b;
  color: white;
}

.menu {
  display: flex;
  gap: 1rem;
}

.menu a { color: white; }

```

```

.conteneur {
  width: min(100% - 2rem, 720px);
  margin: 2rem auto;
}

img {
  max-width: 100%;
  height: auto;
  border-radius: 12px;
}

form {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

input, textarea, button {
  font: inherit;
  padding: 0.6rem 0.8rem;
}

button {
  background: #1f6f5b;
  color: white;
  border: 0;
  border-radius: 8px;
  cursor: pointer;
}

.pied {
  text-align: center;
  padding: 2rem;
  color: #555;
}

@media (max-width: 600px) {
  .entete { flex-direction: column; gap: 0.75rem; }
  .menu { flex-direction: column; align-items: center; }
}

```

Criteres de reussite

- Ca s'ouvre sans erreur visible
- Le menu saute aux sections (`#apropos` etc.)
- Lisible sur petite largeur
- Tes couleurs a toi (change le vert si tu veux)

Bonus

- Ajoute une deuxieme page `projets.html`
- Relie-les avec des liens
- Mets 3 "projets" inventes dans une liste

Quand c'est fait : tu as une vraie mini vitrine.

Pas parfaite. Mais a toi. C'est le plus important.

Chapitre 14 - Ce qu'il faut retenir (+ la suite)

Presque au bout. Encore un quiz juste apres, puis on cloture.

En une minute

- HTML = structure (les pieces de la maison)
- CSS = style (peinture et deco)
- Balises = etiquettes (`h1` , `p` , `a` , `img` ...)
- CSS = selecteur + proprietes
- Boites = margin, border, padding, contenu
- Flexbox = ranger les blocs
- Viewport + media queries = ok sur telephone

Petites habitudes qui changent tout

1. Indente ton code (lisibilite)
2. Ferme tes balises
3. Mets toujours un `alt` sur les images
4. Relie un vrai fichier `.css`
5. Teste souvent (modifie, sauvegarde, F5)
6. Une erreur ? Regarde la derniere chose que tu as changee

Erreurs normales

- Oublier le `<link>` vers le CSS
- Mauvais chemin d'image (`src`)
- Typo dans un nom de classe
- Oublier le viewport

Ca arrive a tout le monde. Le debug, c'est juste de la chasse au detail.

La suite (quand tu seras chaud)

- Un peu plus de CSS (grid, animations legeres)
- JavaScript pour rendre la page interactive
- Accessibilite (clavier, contrastes, lecteurs d'ecran)
- Mettre ton site en ligne

Mais d'abord : le quiz. Puis tu pourras vraiment dire "c'est fait".

QUIZ

Quiz final - Teste-toi !

Avant de dire "j'ai fini", on joue un peu.

Lis la question. Cherche la reponse dans ta tete.

Les corriges sont juste apres. Pas de triche... enfin, si tu veux. C'est ton livre.

Questions

Q1. HTML, ca sert surtout a quoi ?

- A) Mettre des couleurs
- B) Structurer le contenu de la page
- C) Remplacer le navigateur

Q2. CSS, ca sert surtout a quoi ?

- A) Donner le style (couleurs, tailles, rangement...)
- B) Creer la base de donnees
- C) Envoyer des emails

Q3. Ou places-tu le vrai contenu visible de la page ?

- A) Dans `<head>`
- B) Dans `<body>`
- C) Dans `<!DOCTYPE html>`

Q4. Quelle balise donne le titre principal (le plus important) ?

- A) `<p>`
- B) `<h1>`
- C) `<br>`

Q5. Pour un lien, quel attribut donne l'adresse ?

- A) `src`
- B) `alt`
- C) `href`

Q6. Sur une image, pourquoi mettre `alt` ?

- A) Pour faire joli
- B) Pour decrire l'image si elle ne s'affiche pas (et pour l'accessibilite)
- C) Pour agrandir l'image

Q7. Dans une feuille CSS, c'est quoi un selecteur ?

- A) Le nom du fichier
- B) La partie qui dit "sur quoi" s'applique le style
- C) Un bouton du navigateur

Q8. Margin et padding, c'est quoi la difference simple ?

- A) Margin = espace dehors, padding = espace dedans
- B) C'est exactement pareil

- C) Padding = couleur, margin = police

Q9. Flexbox, ca sert a quoi ?

- A) Compresser les images
- B) Ranger des blocs facilement (ligne ou colonne)
- C) Creer un mot de passe

Q10. Quelle balise meta aide le site sur telephone ?

- A) `charset`
- B) `viewport`
- C) `stylesheet`

Q11. Ou mets-tu en general le `<link rel="stylesheet" ...>` ?

- A) Dans le `<head>`
- B) Apres `</html>`
- C) Dans une image

Q12. Une liste a puces, c'est surtout :

- A) `<ol>` + `<li>`
- B) `<ul>` + `<li>`
- C) `<table>` + `<td>`

Reponses

1. **B** - HTML structure.
2. **A** - CSS style.
3. **B** - le `<body>` .
4. **B** - `<h1>` .
5. **C** - `href` .
6. **B** - description / accessibilite.
7. **B** - le "sur quoi".
8. **A** - dehors / dedans.
9. **B** - ranger les blocs.
10. **B** - viewport.
11. **A** - dans le `<head>` .
12. **B** - `<ul>` + `<li>` .

Score rapide

- 10 a 12 : nickel. Tu as les bases.
- 7 a 9 : bien. Relis juste 2-3 chapitres faibles.
- moins de 7 : normal. Reprends les chapitres 2, 8 et 10. Puis reviens.

L'important c'est pas d'etre parfait du premier coup.

C'est de comprendre un peu plus a chaque essai.

Bravo.

Bravo. Tu es alle au bout. C'est deja une vraie victoire.

Serieusement : bravo.

Tu es alle au bout d'un vrai premier livre sur le web.

Pas juste "j'ai regarde une video". Tu as lu. Tu as essaye. Tu as (surement) plante. Puis tu as continue.

C'est exactement comme ca que ca marche.

Ce que tu sais faire maintenant

- Creer une page HTML propre
- Ajouter texte, liens, images, listes, formulaires
- Habiller avec du CSS
- Ranger avec Flexbox
- Penser un minimum au telephone

C'est deja enorme pour un debut.

Petite mission (si tu veux)

Reprends ta page perso du chapitre 13.

Ameliore juste 3 trucs :

1. une meilleure couleur
2. une image avec un vrai `alt`
3. un menu qui marche bien sur petite largeur

Montre-la a quelqu'un.

Meme a ton chat. Le chat jugera pas.

La suite, quand tu seras chaud

JavaScript. Pour rendre la page un peu vivante.

Mais pas ce soir si tu es fatigue. Repose-toi. Tu l'as merite.

Encore bravo.

Et a bientôt pour la suite.

Tu as les briques. Maintenant, construis.